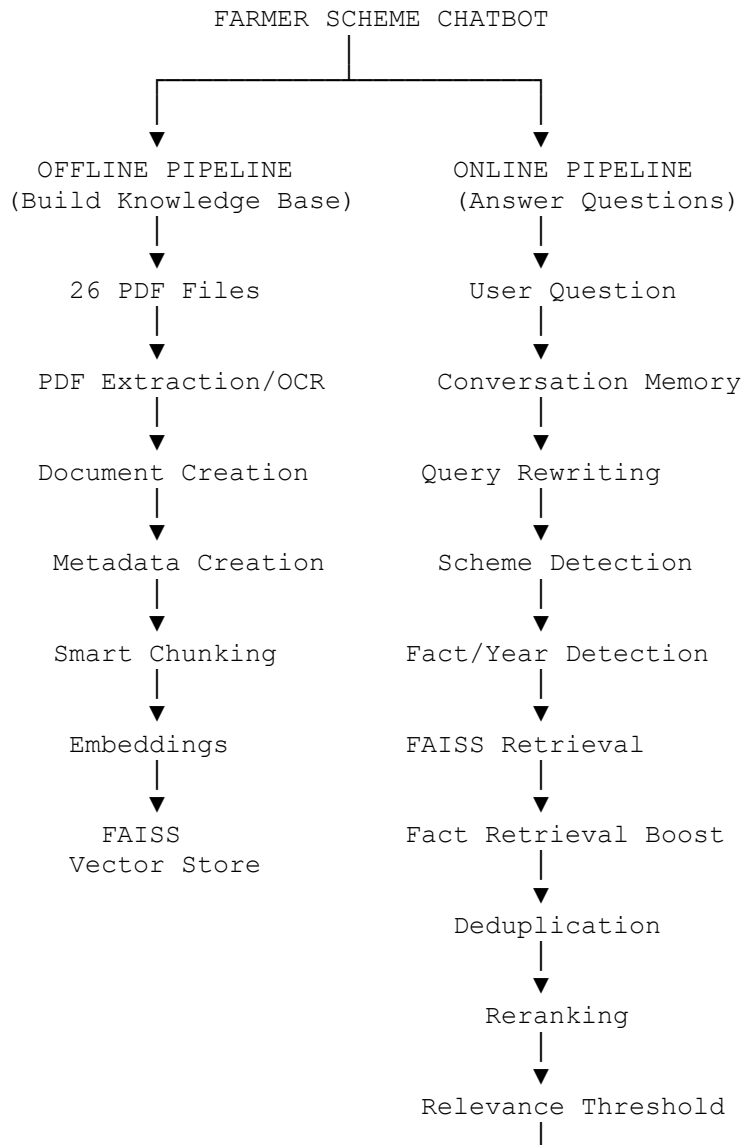


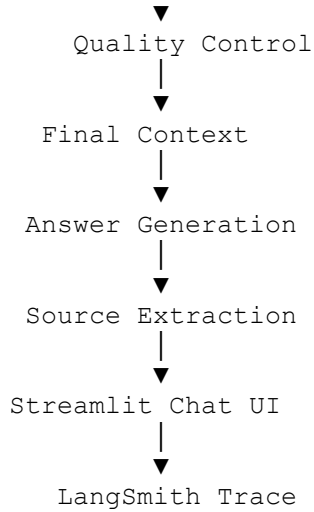
FARMER SCHEME AI CHATBOT

Based on the RAG system we have built and tested so far, the project can be understood as **two major pipelines**:

1. **Offline Knowledge-Base Pipeline** — PDFs → OCR → Documents → Chunks → Embeddings → FAISS
 2. **Online Question-Answer Pipeline** — User question → Rewrite → Scheme routing → Retrieval → Reranking → Answer → Sources
-

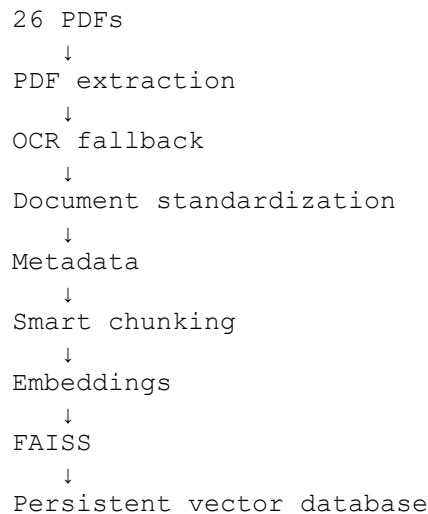
1. Overall Architecture





2. OFFLINE PIPELINE — Knowledge Base Creation

This pipeline runs when you want to **build/rebuild your vector database**.



Step 1 — PDF Loading / OCR

Your source documents are:

```
data/pdf/
├── Agriculture Infrastructure Fund.pdf
├── Animal Husbandry Infrastructure Development Fund.pdf
├── ...
└── Vibrant Villages Programme.pdf
```

The system:

- loads PDF pages
- attempts normal PDF text extraction
- identifies poor/empty pages
- applies OCR where required
- validates OCR quality
- produces page-level documents

You have already successfully processed:

26 PDFs
90 pages
221 final chunks

3. Document Standardization

Raw extracted/OCR text is converted into LangChain:

Document

Each document contains:

page_content
metadata

Example:

```
Document
├── page_content
│   └── "Livestock Insurance..."
└── metadata
    ├── file_name
    ├── page
    ├── document_type
    ├── content_type
    ├── section
    ├── activity_number
    ├── item_number
    ├── parser
    ├── extraction_method
    └── ...
```

This is important because your chatbot isn't just searching text — it also knows **where the text came from**.

4. Metadata Architecture

Your metadata layer provides information such as:

```
source
file_name
document_type
page
page_number
section
content_type
activity_number
activity_marker
item_number
ocr_confidence
has_ocr_warning
parser
extraction_method
language
version
raw_description
raw_cost
```

This enables things like:

```
Which scheme?
Which PDF?
Which page?
What type of content?
OCR or normal extraction?
Which activity?
Which cost item?
```

5. Smart Chunking

You don't blindly split every page into fixed-size pieces.

Your chunking logic recognizes different content types:

```
activity
cost_norm
paragraph
table
heading
list
other
```

Some content is kept intact:

```
activity
cost_norm
heading
```

while long content can be split.

Current configuration:

```
CHUNK_SIZE      = 800
CHUNK_OVERLAP   = 120
```

This improves retrieval because an important scheme fact isn't unnecessarily separated from its context.

6. Embedding Pipeline

Each chunk is converted into a numerical vector.

Current embedding model:

```
BAAI/bge-small-en-v1.5
```

Configuration:

```
Dimension       : 384
Device          : CPU
Normalized      : True
```

Conceptually:

```
Text chunk
  ↓
Embedding Model
  ↓
[0.12, -0.08, 0.31, ...]
  ↓
384-dimensional vector
```

7. FAISS Vector Store

The vectors are stored in:

```
data/vector_store/faiss_index/
```

FAISS allows the chatbot to find chunks semantically similar to the user's question.

For example:

```
User:
"What is PM-KISAN?"

      ↓

Embedding

      ↓

FAISS similarity search

      ↓

Relevant PM-KISAN chunks
```

8. ONLINE PIPELINE — User Question

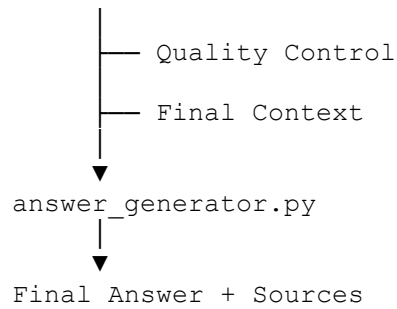
This is the **most important execution flow**.

When the user asks:

How much livestock insured in financial year (2024-25)?

the system executes approximately:

```
User Question
      ↓
query_pipeline.py
├── Conversation Memory
├── Query Rewriter
├── Scheme Router
├── Fact/Year Detection
├── FAISS Retrieval
├── Fact Retrieval Boost
├── Deduplication
├── Reranking
└── Relevance Threshold
```



9. Conversation Memory

File:

`conversation_memory.py`

Purpose:

User:

What is PM-KISAN?

Bot:

PM-KISAN provides income support...

User:

What are its benefits?

The second question doesn't explicitly say PM-KISAN.

Memory allows the system to understand the conversation context.

Current configuration:

`MAX_TURNS = 5`

10. Query Rewriting

File:

`query_rewriter.py`

Example:

Previous:

What is PM-KISAN?

Current:
What are its benefits?

gets transformed conceptually into:

What are the benefits of PM-KISAN?

This rewritten question is then used for retrieval.

Important distinction:

Original Question
↓
Query Rewriter
↓
Standalone Question
↓
Retrieval

The system still remembers the original user question for the final response.

11. Scheme Router

File:

scheme_router.py

This identifies the scheme mentioned in the question.

Example:

"What is PM-KISAN?"

↓

Scheme:
Pradhan Mantri Kisan Samman Nidhi

↓

Target:
Pradhan Mantri Kisan Samman Nidhi.pdf

For your latest test:

How much livestock insured in financial year (2024-25)?

the router detected:

Scheme:
Livestock Insurance

Target:
Livestock Insurance - An introduction.pdf

This prevents unrelated scheme documents from dominating retrieval.

12. Fact / Year Detection

This is one of the improvements we recently added.

The system identifies questions such as:

How much...?
How many...?
What percentage...?
What amount...?
What was the number...?
During financial year...?

as fact-oriented questions.

Then it generates additional retrieval queries.

For example:

Original:
How much livestock insured in financial year (2024-25)?

Additional queries:

Livestock Insurance total number amount percentage beneficiaries insured
coverage

Livestock Insurance livestock insured total number

Livestock Insurance animals insured number coverage

This significantly improves retrieval of exact numbers.

13. FAISS Retrieval

File:

retriever.py

The system initially retrieves:

20 documents

Then additional fact/year queries can retrieve more.

Your recent test:

Base documents retrieved: 20
Extra documents retrieved: 60

Then:

80 documents
↓
Deduplication
↓
33 documents

14. Reranking

File:

reranker.py

Initial retrieval is based on embedding similarity.

Reranking asks a stronger model:

"How relevant is this document to this exact question?"

Current reranker:

BAAI/bge-reranker-base

For your livestock question:

Page 1
Rerank score = 0.990910

That was exactly the correct page.

15. Fact Boosting

After reranking, the system adds additional relevance for documents containing:

- requested scheme
- requested year
- factual keywords
- requested terms

Your latest example:

```
Page 1
Rerank score = 0.990910
Fact boost   = 0.25

Effective score = 1.240910
```

So the correct document remains at the top.

16. Relevance Threshold

Documents below the configured relevance threshold are removed.

Conceptually:

```
33 documents
  ↓
Reranking
  ↓
Threshold
  ↓
Relevant documents
```

This prevents weak documents from reaching the LLM.

17. Quality Control

The pipeline then performs another filtering stage.

Retrieved

↓
Reranked
↓
Threshold
↓
Quality Control
↓
Final Context

Your livestock example ended with:

Documents after threshold: 3
Documents after quality control: 3

18. Final Context

The answer generator receives the strongest documents.

For your latest question, the critical context was:

Livestock Insurance - An introduction.pdf
Page 1

21.01 Lakh
Livestock insured in financial year (2024-25)

This is exactly the evidence the LLM needs.

19. Answer Generation

File:

answer_generator.py

This is the final LLM stage.

The LLM receives:

User question
+
Retrieved context
+
Grounding instructions

For factual questions, we specifically instruct it:

VALUE + ENTITY + TIME PERIOD

must correspond to the user's question.

This prevented the previous error:

✗ 21.01 Lakh households...

and produced:

✓ 21.01 Lakh livestock were insured in financial year (2024-25).

20. Source Attribution

The answer generator also extracts:

File
Page
Content type
Rerank score

So the UI can show:

Sources

Livestock Insurance - An introduction.pdf
Page 1

This is very important for a government-scheme chatbot because users should be able to see **where the answer came from.**

21. Streamlit UI

Your frontend is:

app.py

Architecture:

Browser
↓
Streamlit
↓
app.py

↓
Query Pipeline
↓
Answer
↓
Streamlit
↓
User

The user doesn't need to see all the internal retrieval steps.

They see something like:

Farmer Scheme Chatbot	
User: How much livestock was insured?	
Bot: 21.01 Lakh livestock were insured in financial year 2024-25.	
Sources: Livestock Insurance.pdf — Page 1	
Ask a question...	▶

22. LangSmith

LangSmith is your **observability/tracing layer**.

It doesn't replace your RAG pipeline.

Instead:

Streamlit
↓
Query Pipeline
↓
LangSmith tracing
↓
Observe:
— question
— rewriting
— retrieval
— reranking
— LLM call
— latency
— errors

└─ final answer

This lets you debug questions such as:

Why did the chatbot give the wrong answer?

You can inspect:

```
Question
↓
Retrieved documents
↓
Reranker scores
↓
Final context
↓
Prompt
↓
LLM response
```

23. Your PY File Execution Map

This is the part you should keep as your **master reference**.

A. OFFLINE / DATA INGESTION

The order is:

```
PDF files
↓
[PDF/OCR ingestion script]
↓
Document standardization
↓
metadata_schema.py
↓
chunking.py
↓
embeddings.py
↓
vector_store.py
↓
FAISS index
```

The important conceptual execution order is:

1. PDF/OCR ingestion
2. Document standardization
3. metadata_schema.py

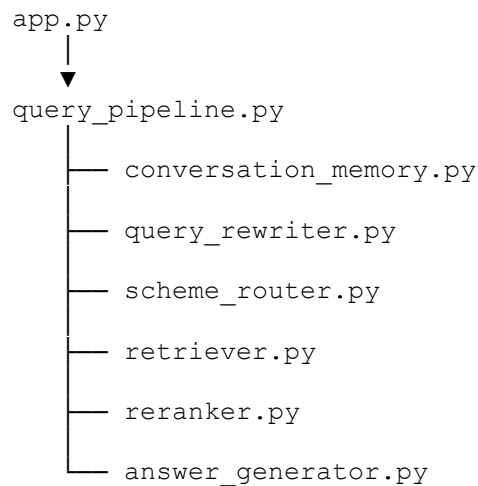
4. chunking.py
5. embeddings.py
6. vector_store.py

These are primarily **build-time components**.

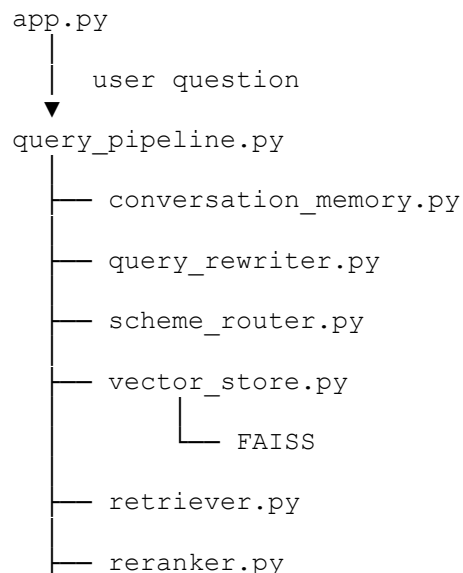
You don't execute all of them every time a user asks a question.

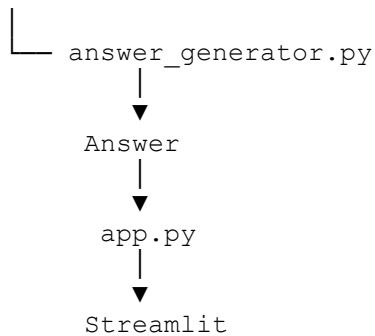
24. ONLINE / CHAT EXECUTION

When the user starts the chatbot:



More precisely:





25. Which `.py` Runs First?

This is an important distinction.

During development / rebuilding the knowledge base:

You run the **ingestion/build script** first.

It creates:

```
data/vector_store/faiss_index/
```

After that, you normally **don't rebuild the index for every question**.

During actual chatbot usage:

You start:

```
streamlit run app.py
```

Then:

```
app.py
  ↓
query_pipeline.py
  ↓
all supporting modules
```

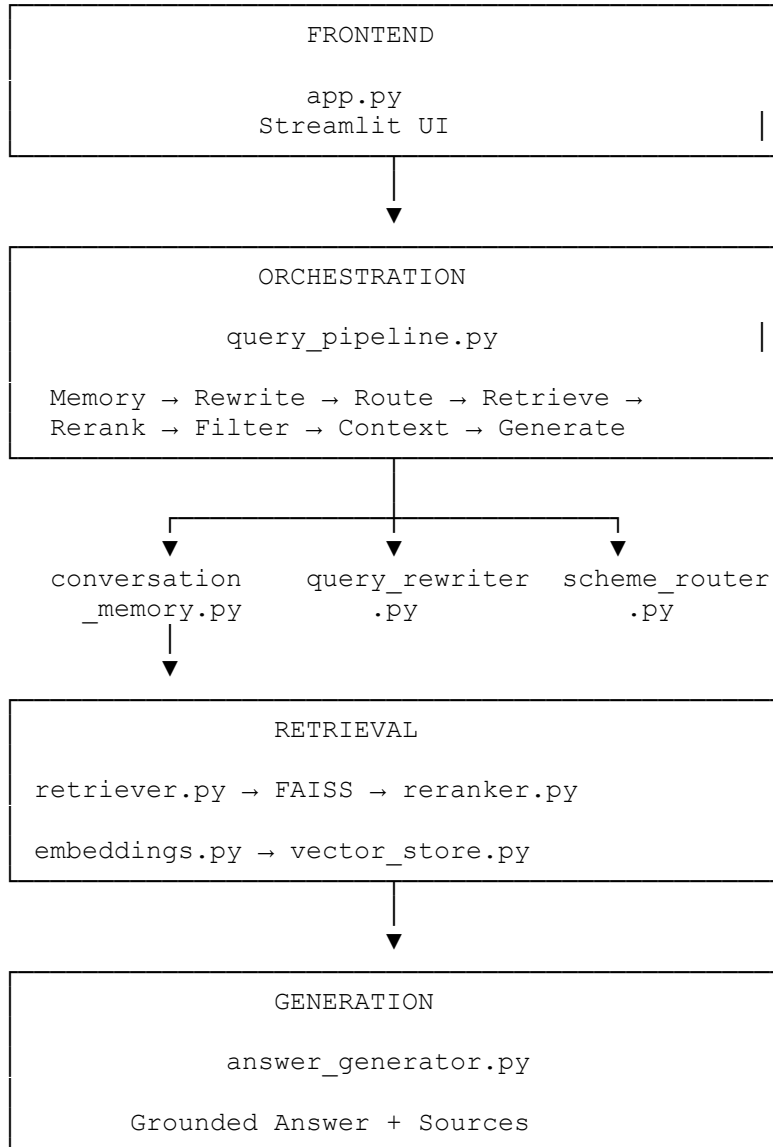
So the user-facing starting point is:

```
app.py
```

not `answer_generator.py`.

26. Complete Project Architecture

I recommend thinking about your project in these 4 layers:



And underneath all of it:

```
data/pdf/  
↓  
OCR / ingestion  
↓  
Documents  
↓  
Chunks  
↓
```

Embeddings

↓
data/vector_store/faiss_index/

27. Requirement Bullet Points — What We Are Building

Your farmer-scheme chatbot requirement can be summarized as:

Knowledge Base

- 26 farmer/government scheme PDFs
- PDF text extraction
- OCR fallback for scanned/image PDFs
- OCR quality validation
- Page-level document creation
- LangChain `Document` standardization
- Structured metadata
- Smart semantic chunking
- Chunk overlap
- BGE embeddings
- FAISS vector database
- Persistent local vector store

Retrieval

- Semantic similarity retrieval
- Initial top-K retrieval
- Scheme-aware routing
- Scheme-specific document prioritization
- Fact/year-aware retrieval
- Multiple retrieval queries
- Deduplication
- BGE reranking
- Relevance threshold
- Quality control
- Final context selection

Conversational RAG

- Conversation memory
- Session-based history
- Context-dependent question rewriting

- Pronoun resolution
- Standalone query generation
- Follow-up question support

Answer Generation

- OpenAI LLM
- Temperature 0
- Grounded generation
- No unsupported facts
- Exact numeric fact extraction
- Entity/value/time matching
- No mixing of unrelated facts
- Concise responses
- Source attribution
- Page-level source references

UI

- Streamlit
- Chat interface
- Conversation history
- User questions
- Assistant answers
- Source display
- Session handling
- Error/debug handling

Observability

- LangSmith tracing
- End-to-end RAG trace
- LLM monitoring
- Retrieval debugging
- Latency monitoring
- Error tracking
- Prompt/response inspection

28. Most Important Design Principle

Your system should **not** be:

PDF → Embedding → LLM

It is much stronger than that.

Your actual architecture is:

